

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	G.Hima Bindu
Name	192511377

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I G.Hima Bindu(192511377) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *G.Hima Bindu*

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1.A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Hadoop job repeatedly fails because input splits are uneven and reducers remain idle

The main problem may be that the input data is not divided properly. In Hadoop, the input file is divided into smaller splits and each split is given to a mapper. If some splits are very large and others are small, some mapper tasks will take more time to finish.

Because of this, some reducers may stay idle and wait for the mapper results. This leads to poor performance and increases the total execution time of the job.

Corrective actions:

- ✓ Check whether the input data is evenly distributed.
- ✓ Use proper input split and block sizes.
- ✓ Avoid having very large input files along with very small files.
- ✓ Select a suitable number of mapper tasks.
- ✓ Use a proper partitioner to distribute data among reducers.
- ✓ Check for data skew in the input keys.
- ✓ Configure the number of reducers according to the workload.
- ✓ Monitor mapper and reducer execution times.

Therefore, the input data should be properly partitioned so that the workload is distributed evenly among all the tasks.

2.An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or Name Node design error.

HDFS shows data unavailability after a single node failure

HDFS is designed to provide fault tolerance by storing multiple copies of data blocks. If data becomes unavailable after only one Data Node fails, there may be a problem with the replication factor.

For example, if a block has only one copy and that Data Node fails, the block cannot be accessed. Another possible problem is that the system has only one Name Node. If the Name Node fails, the HDFS system may also become unavailable.

Corrective actions:

- ✓ Set a suitable replication factor, such as 3.
- ✓ Store replicas on different Data Nodes.
- ✓ Use rack-aware replication.
- ✓ Check for under-replicated blocks regularly.

- ✓ Enable Name Node High Availability.
- ✓ Maintain Active and Standby Name Nodes.
- ✓ Monitor the health of Data Nodes.
- ✓ Take regular backups of important data.

Thus, proper replication and Name Node High Availability are important to make HDFS continue working even when a node fails.

3.A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

ETL and Apache Ni Fi workflow creates duplicate records

The main reason for duplicate records may be that the same data is processed more than once. In Apache NiFi, a Flow File can be processed again when a failure or retry occurs. If there is no proper tracking mechanism, the same record may reach the analytics database multiple times.

Duplicates can also occur when two different ingestion paths send the same data to the database. Another reason can be the absence of a unique key in the target database.

Corrective actions:

- ✓ Use proper state management in Ni Fi.
- ✓ Give each record a unique ID.
- ✓ Check the NiFi data flow and avoid duplicate paths.
- ✓ Use deduplication before storing the data.
- ✓ Add primary key or unique constraints in the database.
- ✓ Use UPSERT or MERGE operations when required.
- ✓ Configure retry and failure handling properly.
- ✓ Make the process idempotent so that repeated processing does not create another record.

Therefore, proper record identification and duplicate checking should be used before inserting data into the analytics store.

4.A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

YARN cluster suffers from resource starvation

The problem is probably related to improper resource scheduling. When many users submit jobs at the same time, one user or one large application may consume most of the CPU and memory resources. Because of this, other applications may have to wait for resources.

YARN should divide the available resources properly among different users and applications. Without proper queues and limits, resource starvation can occur.

Corrective actions:

- ✓ Configure the YARN Capacity Scheduler or Fair Scheduler.
- ✓ Create separate queues for different users or departments.
- ✓ Set CPU and memory limits for queues.
- ✓ Set limits on the number of applications submitted by a user.
- ✓ Give higher priority to important jobs when necessary.
- ✓ Monitor CPU and memory usage.
- ✓ Configure container resources properly.
- ✓ Regularly check Resource Manager and Node Manager status.

Thus, proper scheduling and resource allocation help all users get a fair share of the cluster resources.

5.A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Backup restores outdated data after recovery

The problem may be that the backup system is not taking backups frequently enough or the recovery process is using an old backup. If replication is delayed, the backup copy may also contain old data.

For example, if the last backup was taken several hours before a failure, all the changes made after that backup may be lost during recovery. Keeping only one backup copy can also create problems.

Corrective actions:

- ✓ Take backups at regular intervals.
- ✓ Use both full and incremental backups.
- ✓ Maintain different versions of backups.
- ✓ Monitor replication delay.
- ✓ Check whether each backup has completed successfully.
- ✓ Store backup copies at different locations.
- ✓ Regularly test the restoration process.
- ✓ Define proper RPO and RTO for the system.

Therefore, maintaining recent and multiple backup versions helps restore the latest valid data after a failure.

